An engineering field guide · July 2026

# Sim-Engine Secret Sauce, Explained

*What six physics engines actually do under the hood — read from their source code — and what that means for a two-robot combat-RL stack that has hit the limits of MJX. Companion to* `notes/sim-engine-secret-sauce.md`*, which carries the full file-and-line citations.*

This project trains four-legged fighting robots: two of them, in one arena, millions of times in parallel on an A100. That workload has produced five very specific complaints against the current MuJoCo-MJX-brax stack — and an itch to build a bespoke simulator that "compiles from a strictly typed language down to the simplest GPU operations." Before building anything, we read the source of every engine that matters. The short version: **the bespoke compiler already exists** (NVIDIA Warp), **the bespoke engine is already being built in the open** (mujoco_warp, Newton), and every one of the five complaints is either fixed there or fixable with an afternoon of model surgery. The long version follows, with diagrams.

> **The thread to hold onto.** A simulator is a promise machine. Every millisecond it asks: where are the bodies, what is touching, which joints must stay together, what forces should exist, and where should everything move next? The drama in this note is not "physics is mysterious." It is that the fastest answer depends on how much of that promise can be decided ahead of time, and how much has to be discovered live while 8,192 little worlds are running side by side on the GPU.

### The five complaints, and where each one ends up

| Pain point | Root cause | Resolution |
|---|---|---|
| Contact cost scales with *possible* pairs, not actual ones | MJX freezes the full pair list into the compiled program (§3) | mujoco_warp compacts to actual pairs with atomics (§5) |
| `mjx.ray` silently ignores heightfields (lidar) | Its dispatch table simply has no HFIELD entry (§3) | Done in mujoco_warp, incl. mesh BVH (§5); port = contribution #2 |
| Slider-crank leg needs dt=0.002 or explodes | Soft `<connect>` = explicit stiff spring at a singular Jacobian (§9) | Quartic joint coupling — zero engine code (§9) |
| JIT compiles take minutes | Whole-pipeline XLA trace, every shape baked in (§3) | Warp: per-module compile, ~5 ms warm cache (§4) |
| brax wrapper bugs | brax sits on the JAX backend of MJX | Direct mjwarp / mjlab / Newton paths (§5–6) |

## 1 · The map: who sits on what

Almost everything in this story is one family. MuJoCo's C engine defines the physics *semantics*; MJX re-expresses them for JAX; mujoco_warp re-implements them as explicit GPU kernels on Warp; Newton wraps mujoco_warp as its primary solver and adds seven more; Isaac Lab rides PhysX today and is

adopting Newton. Genesis stands apart on its own compiler fork. Everything green-to-aqua below is Apache-2.0 open source.

> **Plain English: "semantics" means the rules of the little universe.** If a foot penetrates the floor, how springy is the correction? If two bodies touch, how much friction exists? If a joint says two points must meet, is that treated as an exact law or a very stiff rubber band? Engines can run on different hardware and still share semantics, just like two calculators can have different chips and still agree that 2 + 2 = 4.

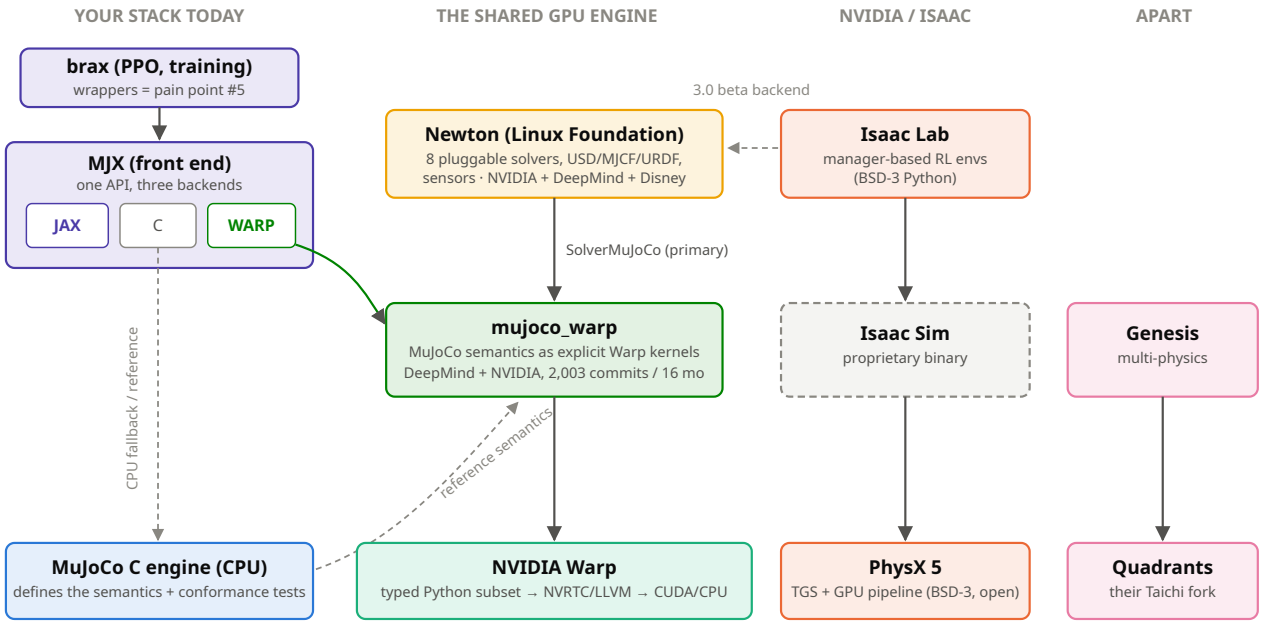


*Fig. 1 — The 2026 GPU-simulation family tree. Solid arrows = "runs on"; dashed = reference or in-progress. The WARP chip inside MJX is the escape hatch: same API, mujoco_warp underneath.*

## Vocabulary in six boxes

> **solref / solimp (MuJoCo).** Every constraint — contact or equality — is a virtual spring-damper whose *impedance* d varies with violation depth and is hard-clamped below 1. Consequence: finite stiffness everywhere, so the solve is a strictly convex optimization with bounded forces; the price is drift (constraints are never exact).

> **PGS vs Newton (solvers).** PGS sweeps constraints one at a time (cheap iterations, slow convergence). MuJoCo's Newton solver descends the whole convex problem with an exact Hessian — few, expensive iterations, made cheap on CPU by sparse factorization plus rank-1 updates when the active set barely changes.

> **TGS (PhysX).** Splits the timestep into sub-steps *inside* the solver: each iteration moves the bodies a little, re-measures constraint error, and integrates springs backward-Euler. Stiffness that would explode an explicit integrator becomes unconditionally stable.

> **XPBD (Newton).** Solves constraints directly at the *position* level each substep with compliance (inverse stiffness) and accumulated multipliers. Also stiffness-unconditionally-stable; accuracy is bought with substep count.

**Featherstone / reduced coordinates.** Parameterize by joint angles so joints are exact by construction — but the mechanism must be a tree. Closed loops need a cut joint plus a constraint (all engines here), or coordinate elimination (none, automatically).

**Dense vs sparse efc.** MuJoCo-family solvers build a constraint Jacobian whose rows are constraint equations. Cost follows *allocated* rows — which is why MJX's static worst-case allocation hurts and mujoco_warp's runtime compaction doesn't.

## High-school math survival kit

**A variable is just a dial.** If x is a joint angle, changing x turns the joint. A function is a machine that reads the dial and returns something useful, like "where is the foot?" or "how far did the slider move?"

**A derivative is slope.** You do not need formal calculus here. Read "d slider / d angle" as: if I nudge the angle a tiny bit, how much does the slider move? At dead center, that slope becomes almost zero, which is the heart of §9.

**A matrix is a table of cause and effect.** A simulator has many dials: hip angle, knee angle, body x, body y, and so on. A matrix stores how each dial affects each constraint. Big tables are not scary; they are spreadsheets with multiplication rules.

**A Jacobian is the simulator's "nudge table."** Each row says how one promise changes when the state changes: foot height, joint alignment, contact distance. If a row loses useful directions, the solver has fewer handles to pull on.

**Quadratic means pairs explode.** Ten parts against ten parts is 10×10 = 100 possible pairs. Twenty against twenty is 400. Double both sides and the work does not double; it roughly quadruples.

**Convex means one bowl.** If an optimization problem is shaped like one smooth bowl, rolling downhill cannot get trapped in a wrong valley. MuJoCo works hard to keep its constraint solve in that friendly category.

## 2 · MuJoCo: why it's fast on a CPU, and why GPUs hate it

MuJoCo rebuilds its entire contact and constraint world from scratch every step — and is fast *because* of that, not despite it. The per-step pipeline:

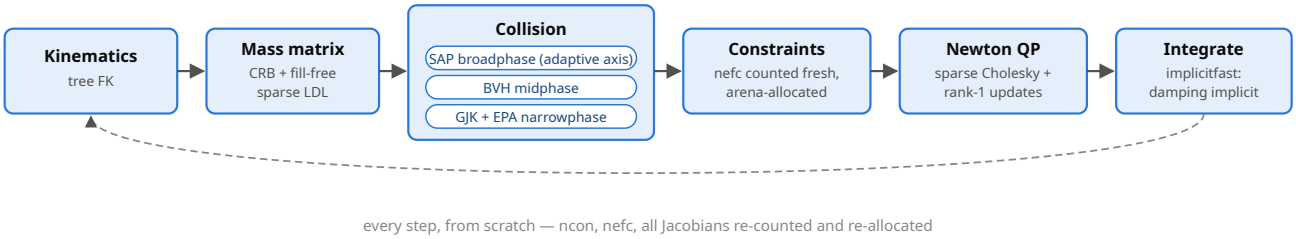


*Fig. 2 — MuJoCo's step (`engine_forward.c mj_fwdPosition`). Nothing persists between steps except the state vector.*

**The contact model.** Broadphase is a sweep-and-prune whose sweep axis *adapts to the scene* — it eigen-decomposes the covariance of all geom centers each step and sorts along the dominant axis. Narrowphase dispatches through a function table: analytic formulas for primitive pairs (sphere, capsule, box…) and an in-house GJK+EPA with polygon-clipped multi-contact for convex shapes. **The constraint model** is the famous soft one: force ≈ −K·d(r)·violation − B·velocity, where impedance d(r) < 1 always, so the regularizer R > 0 keeps the optimization strictly convex — *constraint forces are finite by construction, even at a singular mechanism Jacobian* (this matters in §9). The Newton solver then converges in a handful of iterations because the problem is convex and its Hessian exact; between iterations it never refactorizes, applying rank-1 Cholesky updates only for constraints whose active state flipped.

> **The spring formula, translated.** Read force ≈ −K·d(r)·violation − B·velocity as: "push back harder when the rule is broken more, and also damp motion that is making the break worse." K is the stiffness knob, B is the shock-absorber knob, and d(r) is MuJoCo's safety dial. Because d(r) stays below 1, the engine never asks for an infinite force. That does not make every mechanism stable at every timestep; it means the math problem the solver sees stays well-posed.

> **Newton, Hessian, Cholesky: three intimidating names for one routine idea.** Imagine the simulator standing on the side of a smooth bowl and trying to reach the bottom. Newton's method uses the local slope and curvature to choose a smart downhill step. The Hessian is the table of curvatures. Cholesky factorization is the fast arithmetic trick for solving the big set of simultaneous equations that says, "given these curvatures, which step should all joints and contacts take together?"

**Why this resists GPU-ification.** Four structural reasons, all visible in the code:

1. **Dynamic shapes everywhere** — contact and constraint counts are recomputed and arena-allocated per step; GPUs want static shapes.

2. **Branchy narrowphase** — a function-pointer table into wildly different algorithms with data-dependent iteration counts; hostile to SIMT warps.
3. **Tree-ordered sparse algebra** — the mass-matrix factorization walks parent chains with loop-carried dependencies.
4. **Stateful active-set logic** — the rank-1-update trick is inherently sequential.

> **Why GPUs hate this shape of work.** A GPU is happiest when thousands of workers follow the same recipe on different data: same loops, same array sizes, same memory layout. MuJoCo on CPU is brilliant because it keeps changing the recipe to match the exact scene this instant. That adaptability saves work on one CPU world, but it is awkward when thousands of worlds must run in lockstep.

Every GPU engine in this guide is one answer to the question: *which of these four do you give up, and what do you pay?*

## 3 · MJX: the quadratic two-robot bill, itemized

MJX maps MuJoCo onto JAX by accepting rule #1 above completely: **every shape is frozen at trace time**. Its collision driver enumerates all statically-possible geom pairs in Python while tracing — nested loops over bodies, filtered only by static rules (excludes, weld, parent-child, contype/conaffinity masks). There is no positional filtering: every surviving pair gets its narrowphase function vmapped over it *every step*, touching or not, and every pair gets contact slots and dense constraint-Jacobian rows allocated in the compiled program. Deactivation happens after the FLOPs are spent, by multiplying rows to zero.

> **The key trade: MJX chooses predictability over selectivity.** JAX/XLA wants to know array shapes before the compiled program runs. MJX gives it that certainty by pretending every allowed pair might matter every step. That makes the program clean to compile, but the robot pays for contact checks between parts that may be meters apart.

> **Why the two-robot case hurts so much.** If robot A has n collision parts and robot B has m collision parts, the possible cross-robot checks are n×m. With 20 and 20, that is 400 before we even talk about the floor. The important word is *possible*: MJX pays for those checks because it froze the list before seeing the live positions.

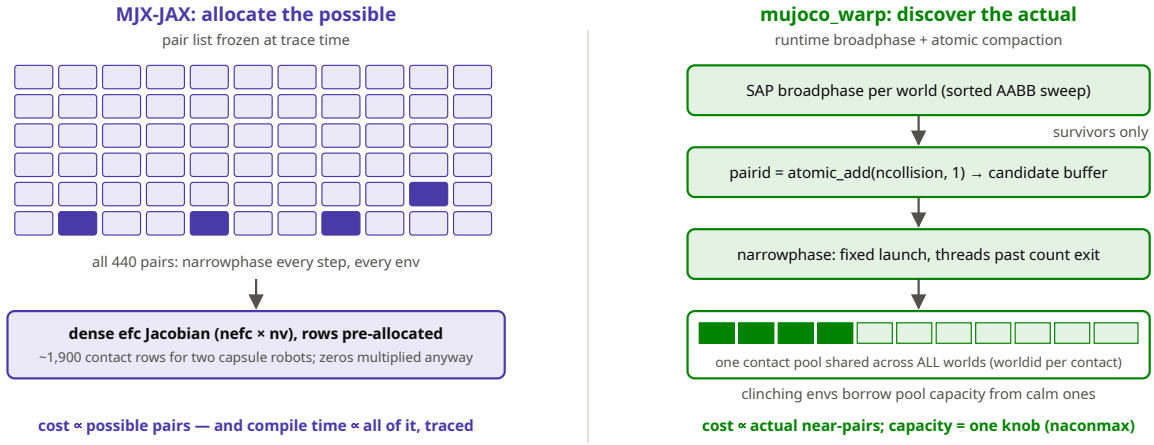


*Fig. 3 — The core architectural difference. Left: MJX evaluates the shaded grid (every statically possible pair) each step; dark cells are the few that actually touch. Right: mujoco_warp's pipeline touches only what the broadphase finds, compacting with atomics into a shared pool.*

**The bill for this project.** Two robots × ~20 collision geoms (torso box, four legs of hip/thigh/calf capsules + foot spheres, optional striker) live in different kinematic trees, so no static filter applies between them: 20×20 = **400 cross-robot pairs** plus ~40 floor pairs. All capsules → ~480 contact slots → at condim=3 pyramidal friction, four Jacobian rows each: **~1,900 dense constraint rows per env-step**, multiplied by 8,192 envs, whether or not the robots are anywhere near each other. The hand-tuned contype/conaffinity masks already in `gen_robot_mjcf.py` (the "F-SPEED" mode) are the only real lever, because they shrink the *trace-time* list itself.

> **What this means in training-budget terms.** PPO does not know or care whether a GPU cycle was spent on a real clinch or on two ghost pairs that were never close. It only sees fewer useful simulated seconds per wall-clock hour. The optimization target is not "make collision elegant"; it is "spend almost every cycle on physically meaningful experience."

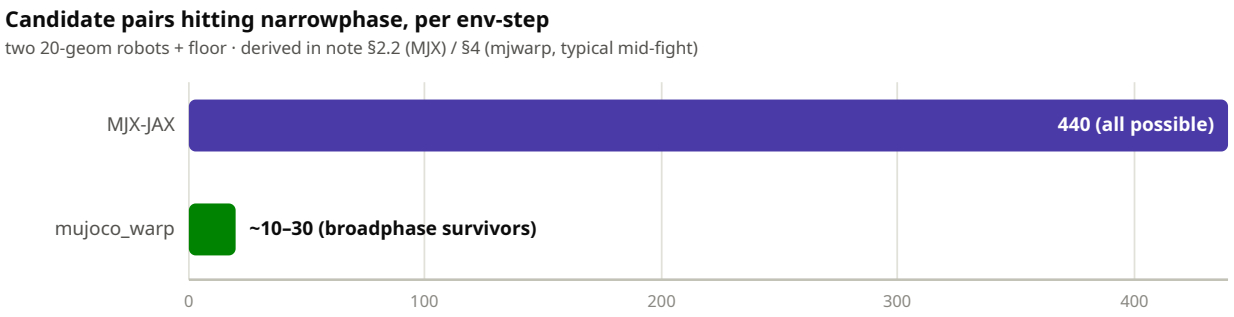


*Fig. 4 — Same scene, same step, ~20× fewer narrowphase evaluations — and the gap widens with geom count (quadratic vs ≈linear).*

**Two more MJX facts worth internalizing.** First, `mjx.ray`'s dispatch table covers exactly six geom types — plane, sphere, capsule, ellipsoid, box, mesh (brute-force, no BVH). Heightfields aren't in the table and there is no error path: a lidar ray aimed at terrain *silently returns "no hit."* Second, the minutes-long JIT is structural, not incidental: the entire collision table, nine constraint builders, the Newton solver with linesearch, and the integrator trace into one XLA graph whose every shape is a constant, and convex colliders specialize per distinct mesh. Nothing inside MJX shortens it; only caching around it helps.

> **The compile wait is a symptom, not a timer problem.** The long JIT is MJX printing a giant custom instruction manual for one exact model shape. Caching can reuse that manual, but it cannot make the manual smaller. The way out is to stop baking every possible pair and solver row into one monolithic compiled graph.

> **If staying on MJX-JAX for now:** keep extending the contype masks; add the experimental `max_geom_pairs` / `max_contact_points` numerics (top-k sphere-distance culling); run Newton with `iterations=1, ls_iterations=4–6`, dense Jacobian — the docs recommend exactly this for RL. And know the escape hatch shipped in the same package: `mjx.put_model(m, impl='warp')` swaps the backend to mujoco_warp under the same API (no autodiff through physics; fine for PPO).

## 4 · Warp: the "strictly typed language → GPU" compiler already exists

The bespoke-simulator dream names a compiler: a strictly typed language lowered to plain GPU kernels. That is literally NVIDIA Warp, five years old, Apache-2.0, DCO-only contributions. A `@wp.kernel` function must annotate every argument (missing annotations are a hard compile error); only 24 whitelisted Python AST node types are accepted — no lambdas, comprehensions, exceptions, recursion, or dynamic data structures; variables cannot change type; in-kernel allocation doesn't exist. What survives those rules compiles to a C++/CUDA source string via literal templates, then through NVRTC (GPU) or an embedded Clang/LLVM (CPU), keyed by a SHA-256 content hash into an on-disk cache.

> **Kernel, in human terms.** A GPU kernel is a tiny job description handed to many workers at once: "thread 0 handle pair 0, thread 1 handle pair 1, ..." Warp's strict Python is useful because it removes ambiguity. If the compiler knows every type and every array shape that matters to one kernel, it can turn that tiny job description into CUDA without asking XLA to understand the whole simulator at once.

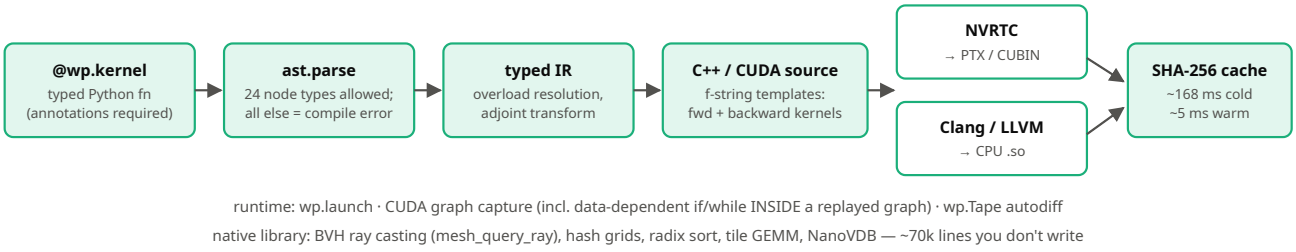


*Fig. 5 — Warp's compile path. Per-module C++ translation units instead of one whole-program XLA graph: that single difference is "seconds vs minutes."*

Scale check: ~7,000 commits, 100 contributors, roughly eight sustained core developers over 5.3 years — order of **20–30 person-years**, most of it in exactly the parts a from-scratch build would need first (adjoint codegen, CUDA graphs, BVH, kernel caching). Writing "a Warp" is not the project; writing *kernels in* Warp is.

> **The leverage shift.** This is the moment the build instinct should change targets. The heroic work is not inventing a private compiler; that mountain already exists and is climbable. The useful work is choosing the few kernels and model changes that make this robot-training loop faster, then sending the general pieces upstream so the next team starts one rung higher.

## 5 · mujoco_warp: MuJoCo semantics rebuilt as explicit kernels

mujoco_warp (DeepMind + NVIDIA, 2,003 commits in 16 months, versioned in lockstep with MuJoCo) answers the four GPU-hostility problems of §2 one by one. Dynamic shapes → fixed-capacity pools filled by atomics, with per-world overflow flags instead of corruption. Branchy narrowphase → GJK/EPA rewritten as Warp device functions dispatched from a static table. Sparse tree algebra → dense tiled Cholesky (small robots) or a sparse path (nv > 32). Stateful solver logic → a CUDA-graph `capture_while` loop in which *each world exits the Newton solve the moment it converges*, decrementing a global counter that ends the loop when all are done. The whole step — collision, solve, integrate — is captured once as a CUDA graph and replayed; a batch of worlds costs one launch.

> **Three GPU tricks, demystified.** A fixed-capacity pool is a parking lot: it has a maximum number of spaces, but the occupied spaces change every step. An atomic add is a take-a-number dispenser, so many GPU threads can safely claim the next free slot without colliding. A CUDA graph is a recorded choreography: once the engine has captured "broadphase, narrowphase, solve, integrate," it can replay that whole dance with very little launch overhead.

The two headline features for this project: **heightfield rays and rangefinder sensors are implemented** (BVH-accelerated mesh rays too — the GitHub issues for hfield ray are all closed), and **domain randomization without recompiles**: Model fields carry a broadcastable leading dimension, so per-world masses, frictions, and gains are an array shape, not a new program. Missing: differentiability (open issue #500), PGS/noslip, plugins.

> **Why this feels different from a rewrite.** The hard promise remains: MuJoCo-like physics, checked against MuJoCo-like expectations. The implementation changes from "one enormous static JAX graph" to "explicit GPU kernels with runtime compaction." That is a much smaller bet than a new physics engine because the rules of the universe are not being reinvented, only the route through the hardware.

> **Caveat before migrating the mesh leg:** open issue **#1270** — connect-constraint anchors don't account for joint reference positions. The slider-crank leg is a ready-made repro; test it first, and see §9 for why you may not need `<connect>` at all. Related: **#1415**, the blocked Cholesky lacks a pivot floor and can NaN near singular configurations — the dead center again.

## 6 · Newton: the bespoke engine, already being built in the open

Newton (Linux Foundation; initiated by NVIDIA, Google DeepMind, and Disney Research; v1.0 in March 2026, ~2,100 commits) is what "build a modern GPU sim on Warp" looks like when twenty funded engineers do it: one Model/State/Control data layer holding *both* maximal and generalized coordinates, with eight interchangeable solver plugins on top and USD/MJCF/URDF importers of unusual fidelity (tendons, heightfields, equality constraints — the latter converted to native loop joints for solvers that want them).

> **How to read "eight solvers."** Think of Newton as one robot description with several physics judges. One judge is MuJoCo-style and careful about MuJoCo semantics. One judge is XPBD and thinks in position corrections. One judge is Featherstone and is excellent for tree-shaped mechanisms but wrong for this closed loop. The power is not that every judge is best; it is that the model layer lets the project choose the judge whose assumptions match the leg.

| Solver | What it is | Relevance here |
|---|---|---|
| **SolverMuJoCo** | full bridge to mujoco_warp (builds an MjSpec, defaults Newton + implicitfast) | "primary backend" — the drop-in path |
| SolverXPBD | position-based, maximal coords | native loop joints, stiffness-stable |
| SolverVBD / AVBD | vertex block descent, ramped penalties | experimental |
| SolverFeatherstone | reduced-coordinate CRBA | **silently ignores loop joints** — never for this leg |
| SolverKamino | Disney's proximal-ADMM NCP solver, *built for kinematic loops + hard contacts* | Beta; the most principled dead-center answer in this survey |
| SemiImplicit / Style3D / ImplicitMPM | legacy rigid / cloth / granular | n/a |

Sensors include a Warp-native tiled-camera raytracer and per-primitive analytic raycast functions — but **no turnkey lidar sensor** (all the pieces exist; composing them is contribution #5 in §11). Governance caveats: committers are overwhelmingly NVIDIA (DeepMind's share flows through mujoco_warp), contributions need a CLA, and several solvers are still flagged experimental. Isaac Lab 3.0 Beta ships Newton as an experimental backend with validated PhysX ↔ Newton policy transfer.

> **The strategic read.** Newton is the closest public artifact to the "bespoke engine" idea, but it comes with governance, tests, importers, and multiple solver experiments already attached. That turns a lonely engine project into a choice: become a user who files sharp repros, or become a contributor whose fixes land where Isaac, MuJoCo, and future labs can all benefit.

## 7 · PhysX 5: the Isaac secret sauce is temporal

PhysX wins stiff-constraint stability a completely different way: **everything is cached, substepped, and re-measured over time** rather than re-derived from scratch. Three mechanisms carry most of it:

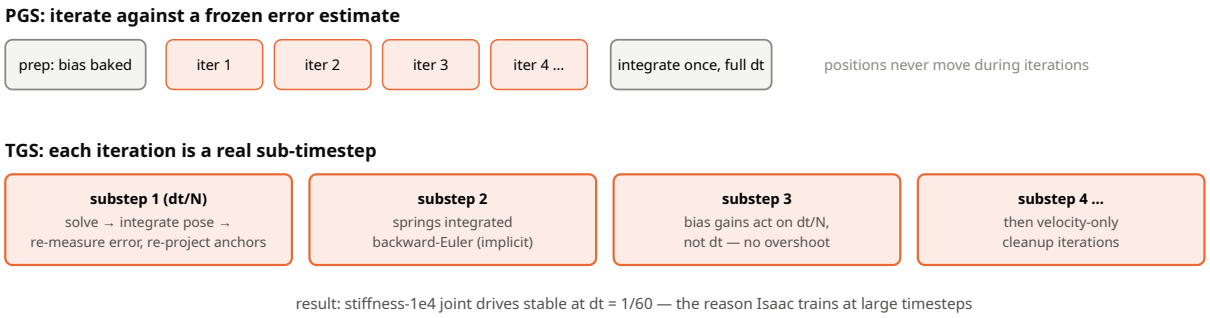


*Fig. 6 — Temporal Gauss-Seidel vs classic projected Gauss-Seidel. TGS re-integrates poses inside the solver loop and solves springs implicitly per substep.*

- **Aggregates:** a whole robot is one broadphase entry with its own local mini-sweep for self-pairs — a 30-link self-colliding robot costs the global broadphase one box.
- **Persistent contact manifolds:** up to four cached contact points per pair, stored in body-local space, refreshed by re-projection each frame and regenerated only when drift exceeds thresholds — most frames do no full contact generation at all.
- **Dynamic GPU contact streams:** an incremental GPU sweep-and-prune reports only found/lost pairs; real contacts bump-allocate into capacity-limited device buffers that warn and drop on overflow. Memory follows expected load, not possible pairs — the same philosophy as mujoco_warp, five years earlier. Environment-ID filtering lives *inside* the broadphase kernel: cross-env pairs are rejected before they exist.

> **TGS without the acronym fog.** Suppose you need to move a stiff joint back into place over 16 milliseconds. A classic solver may estimate the correction once and shove for the whole 16 ms. TGS instead does several smaller real steps: correct a little, move the body, measure again, correct again. Four small nudges usually behave better than one big shove, especially when springs are stiff.

Isaac Lab adds config-driven MDP managers, cloning with env-ID isolation, PhysX contact-report sensors, tiled cameras — and a ray-caster that is just Warp's `mesh_query_ray` against a *single static mesh*: fine for terrain height-scans, useless for sensing the opponent robot. All of Isaac Lab's Python is BSD-3, but it imports the proprietary Isaac Sim binary to run; PhysX itself is BSD-3 with its GPU kernels now open in-tree.

## 8 · Genesis: an honest read

Under the hood, Genesis is a Taichi program whose team now maintains the compiler themselves: upstream Taichi stalled, so they forked it (June 2025) and ship it as "Quadrants." The rigid-body solver is an *openly acknowledged* GPU-batched MuJoCo reimplementation — solref/solimp tuples, CG/Newton solvers, EPA ported line-for-line (the source cites the MuJoCo C file it mirrors), MuJoCo-consistency tests in CI, and the same connect/weld/joint equality trio. It has a real LBVH lidar sensor and Isaac-Gym-derived terrain tools.

> **How to read benchmark claims.** Batched simulation numbers are like miles per gallon on a test track: real, but only for that track. Contact-light scenes with self-collision off can make any GPU simulator look heroic. Robot combat is the opposite: many contacts, changing pairs, sensors, and solver stress. The honest question is not "what was the largest headline number?" but "does the benchmark resemble the pain we actually have?"

The December 2024 launch claimed "430,000× faster than realtime" and "10–80× faster than existing GPU sims." The number was real batched throughput for a contact-light, mostly-static scene (30k Franka envs, one substep, self-collision off); under corrected settings it dropped ~150×, and on contact-rich manipulation Genesis measured 3–10× *slower* than ManiSkill-class sims. The authors later published corrected open-script benchmarks and removed all speed claims from the README (May 2026). Today it is professionally maintained (a full-time team under the Genesis AI company; ~1,500 commits), Apache-2.0 with no CLA, and its genuine differentiator is multi-physics breadth (MPM, SPH, FEM, cloth), not rigid-body speed. Structural risk: a small team owning an entire compiler fork.

## 9 · The toggle problem: the dead center, engine by engine

Each leg of the mesh robot closes a slider-crank loop with a MuJoCo `<connect>` weld between the blade and the pushrod. At top-dead-center ($\varphi = 0$) the crank and conrod are collinear: the loop Jacobian goes singular, the constraint force direction flips as the piston reverses, and at dt = 0.004 the simulation measured 10 m of slide error and $|qacc| \approx 7 \times 10^9$. At dt = 0.002 it tracks the closed form to under a millimeter — hence the whole fleet paying double the steps.

> **Dead center without calculus.** Picture a bicycle pedal exactly at the top of its circle. For an instant, pushing down does not move the bike forward efficiently; the mechanism is lined up in an awkward direction. The slider-crank has the same kind of moment. Near $\varphi = 0$, a tiny crank-angle change produces almost no slider motion, then the direction of useful force flips. That "almost no motion, then flip" is what the singular Jacobian is saying.

> **A Jacobian is not magic; it is a nudge table.** In this leg, one column asks "if the crank turns a little, where does the weld point move?" and another asks "if the slider moves a little, where does the weld point move?" At dead center, one of those nudges stops helping in the direction the constraint cares about. The solver still has numbers, but the table has become a bad handle.

**Why it explodes even though MuJoCo's forces are "bounded."** The per-step convex solve does guarantee a finite force (§2). But that force is applied *explicitly across the step* — the constraint enters the integrator as a plain right-hand-side force, outside the implicit treatment implicitfast reserves for damping. So the soft connect behaves as a stiff discrete-time oscillator; near TDC its effective inertia collapses (50–80 g linkage bodies, near-singular J) while the regularizer that is supposed to match — scaled from inverse weights precomputed *at the reference pose* — no longer fits the configuration. Halving dt restores sampling of the force-direction flip. (Code facts verified; the instability chain is analysis.)

> **"Finite" is not the same as "gentle."** A finite hammer blow can still break a small mechanism if it arrives at the wrong time. MuJoCo prevents the constraint solve from asking for infinity, but the explicit step still samples a fast-changing force too coarsely at dt = 0.004. Halving the timestep is like filming a fast event at twice the frame rate: the flip is no longer skipped.

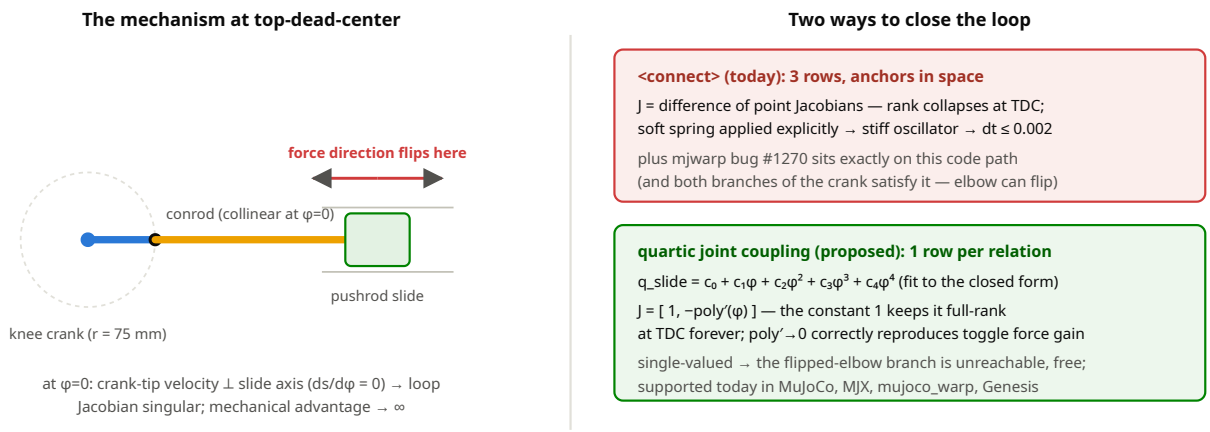


*Fig. 7 — The dead-center problem and the model-level fix: replace the spatial weld with polynomial couplings between the joints themselves.*

**Quartic decoded.** A quartic is just a bendy curve: $q_{slide} = c_0 + c_1\varphi + c_2\varphi^2 + c_3\varphi^3 + c_4\varphi^4$. The c values are fitted from the known slider-crank geometry, the same way a spreadsheet can fit a trendline. Instead of telling the simulator "keep these two 3D anchor points welded," we tell it "when the crank angle is this, the slider position should be that."

**Why the quartic avoids the singularity.** The new constraint directly compares two joint coordinates: slider position minus predicted slider position. Its Jacobian contains a constant 1 for the actual slider coordinate. That means the solver always has one good handle to pull, even when the crank's slope goes flat at dead center. The mechanism still has the right leverage behavior, but the solver no longer closes the loop through a fragile spatial weld.

**The afternoon experiment:** fit the quartic against the closed-form `slider_crank_s(φ)` already in `gen_mesh_robot_mjcf.py` (weight the TDC neighborhood), one coupling for the slide and one for the toe hinge, delete the `<connect>`, and retry dt = 0.004. Gates: fit residual < 0.5 mm over the working range; the loop-consistency contract test; the zero-g whip test; and the toggle-press force profile against the ~1.2 kN connect response. If the quartic can't hold tolerance, MuJoCo's tendon equality allows piecewise shaping.

## How each engine treats a dead-center mechanism

| Engine | Loop closure | Dead-center behavior |
|---|---|---|
| MuJoCo / MJX / mjwarp | soft equality (acceleration level) | forces finite, positions drift; explicit across steps → dt-limited near the singularity (the measured blowup) |
| PhysX TGS | maximal joint between articulation links | position-level + implicit springs per substep: no global dt cut, but the loop joint is documented as the largest error accumulator; mimic joints are linear-gear only |
| Newton XPBD | native loop joints, compliance + multipliers | stiffness-unconditionally stable; accuracy bought with substeps; approximate momentum |
| Newton Kamino | NCP via proximal ADMM, built for loops | the principled answer; Beta — watch it |
| Newton Featherstone | **silently not enforced** | trap — never for this leg |
| Genesis | MuJoCo equality trio | same as MuJoCo |
| *Coordinate elimination* | bake ψ(φ), s(φ) into kinematics | exact, singularity-free, no constraint at all — nobody does it automatically; the quartic coupling is its 95% stand-in, available today |

> **The lesson hiding in the leg.** This is not a story where "physics engines cannot handle our robot." It is a story where a one-dimensional mechanism was encoded as a three-dimensional promise, then punished at the one pose where that promise becomes hardest to interpret. The most powerful change may be the least glamorous one: describe the machine in the coordinates it actually lives in.

## 10 · Licenses, plainly

| Repo | License | To contribute |
|---|---|---|
| MuJoCo + MJX | Apache-2.0 | Google CLA (one-time signature) |
| mujoco_warp | Apache-2.0 | Google CLA; friendly tooling, expects external PRs |
| Warp | Apache-2.0 | DCO only (`git commit -s`) — lowest friction |
| Newton | Apache-2.0 | CLA (Linux Foundation governance) |
| PhysX | BSD-3 | DCO; GPU kernels now open in-tree |
| Isaac Lab | BSD-3 | DCO — but runtime needs proprietary Isaac Sim |
| Genesis | Apache-2.0 | plain PRs, no CLA/DCO |

For a project that prefers MIT: **for using these engines, Apache-2.0 and BSD-3 are practically identical to MIT** — permissive, no copyleft, commercial and closed use fine, and your own code stays MIT. The differences that exist are mostly in your favor: Apache-2.0 carries an explicit patent grant (protection you *want* from NVIDIA- and Google-owned code) plus a keep-the-LICENSE/NOTICE-files obligation; BSD-3 adds only a don't-use-our-name clause. You cannot relicense *their* files as MIT, but you can depend on them, vendor them, or ship them inside an MIT project with attribution.

> **The non-lawyer version.** These licenses do not block the mission. They mostly say: keep notices, do not pretend you wrote their files, and respect contribution paperwork when sending patches back. For this project, the license question should not decide the architecture; performance, correctness, and community leverage should.

## 11 · Verdict: contribute, don't rebuild

**(a) What "bespoke" actually costs.** The compiler half is solved and reusable (Warp, §4). The engine half, measured on a team *re-implementing known semantics with a conformance oracle*, is 10–15 person-years and counting — and still missing autodiff. A from-scratch engine without a reference implementation buys back none of that.

> **The moral is not "dream smaller."** It is "put the dream where it moves the most people." A private simulator would absorb years before it teaches the robot anything new. An upstream heightfield ray, a connect-anchor fix, a better singularity test, or a clean lidar sensor becomes infrastructure other teams can stand on. That is how a local pain point turns into a public tool.

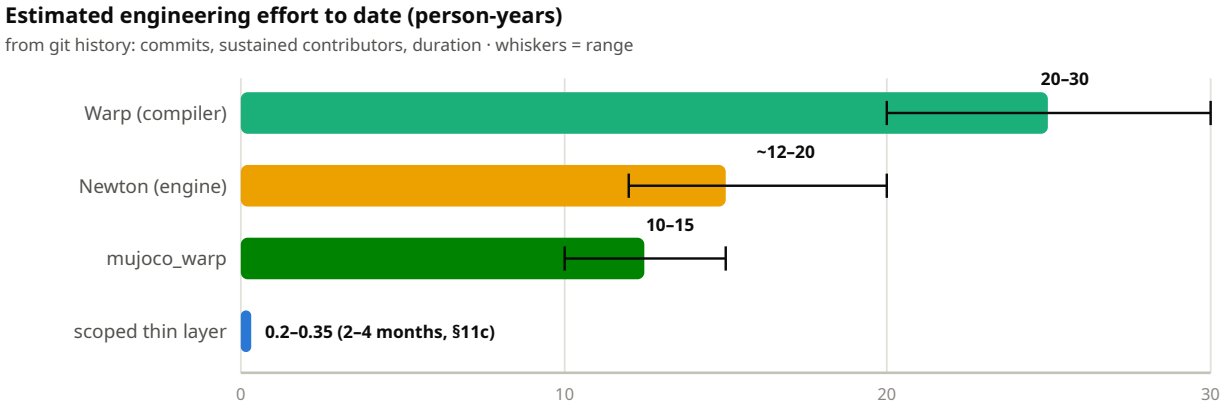


*Fig. 8 — The asymmetry that decides the build-vs-contribute question. (PhysX, at 18+ years of development, doesn't fit on this axis.)*

### (b) The pragmatic ladder.

> **Read the ladder as a sequence of bets.** Each rung has a cost, a measurement, and a stopping rule. Switch the backend and measure. Replace the fragile loop model and measure. Upstream the missing pieces and measure. Only build a thin bespoke layer if profiling shows the remaining bottleneck is truly outside the shared engines.

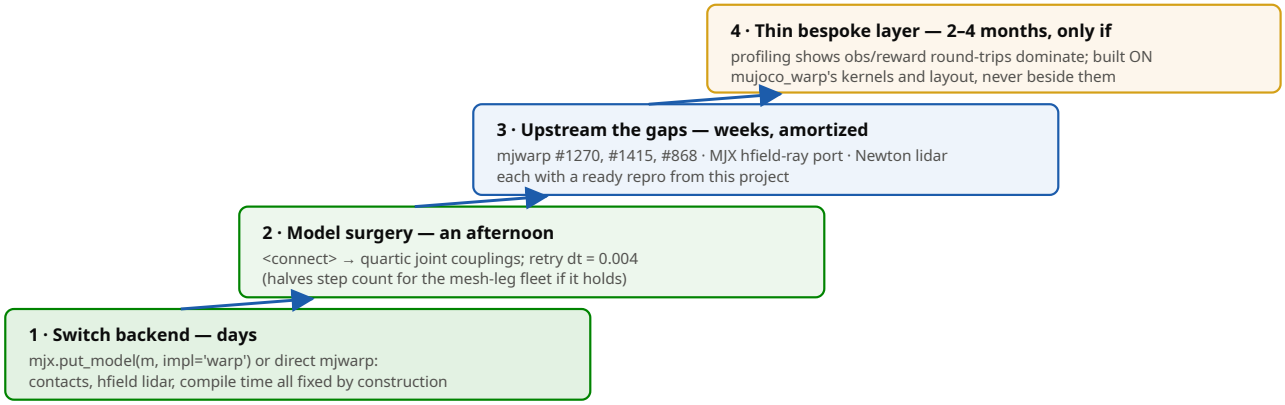


*Fig. 9 — Each rung is independently useful; stop climbing when the training run is fast enough.*

The five named contribution targets, with the files they touch:

1. **mjwarp #1270** (connect anchors vs joint reference positions) — `mujoco_warp/_src/constraint.py`, regression test against the leg's closed form. De-risks the GPU migration of the mesh robot.
2. **MJX heightfield ray** — port `mujoco_warp/_src/ray.py ray_hfield` into `mjx/_src/ray.py`'s dispatch table (and add a warning for unsupported types). Unblocks lidar-over-terrain in the current brax stack.
3. **mjwarp #1415** (Cholesky pivot floor) — `mujoco_warp/_src/block_cholesky.py`. Solver stability exactly at near-singular configurations like TDC.
4. **mjwarp #868** (incremental Hessian) — port the C engine's rank-1 update trick (`engine_solver.c HessianIncremental`) to `_src/solver.py`. Big win for contact-rich two-robot worlds.
5. **Newton lidar sensor** — compose `newton/_src/geometry/raycast.py` into a `SensorLidar` next to the tiled camera. Fills a documented gap.

**(c) If the thin layer is ever built**, its genuinely bespoke 10% is four small things: an exact loop-coordinate joint (ψ(φ), s(φ) baked into kinematics — ~200 lines, the one thing no engine offers), a hand-curated static pair table (~60–120 pairs with analytic capsule kernels — at this scene size, curated-static beats any broadphase), a 144-ray lidar kernel (~200 lines), and obs/reward kernels fused into the captured CUDA step graph (the actual payoff: no device↔host round-trip per control step). Everything else — compiler, contacts, solver, integrator, rays, autodiff — is reuse. Milestones with kill-criteria are in the companion note, §10c.

> **The motivating version.** The world does not need one more isolated robotics stack that only its authors can maintain. It needs shared simulation tools that make ambitious embodied systems cheaper to test, safer to break, and easier to improve. This project has unusually sharp repro cases: two-robot contact load, terrain lidar, a real dead-center mechanism, and a training loop that punishes wasted GPU cycles. Turning those repros into fixes is a way to make the whole ecosystem better while making this robot faster.

*One line to remember: the bespoke compiler exists, the bespoke engine is being built in the open by twenty funded engineers, and this project's genuinely novel needs total a few hundred lines on top of that stack —*

*not a simulator instead of it.*

---

Companion note with full file-and-line citations: `notes/sim-engine-secret-sauce.md` (commit-pinned: mujoco 14c0b0c9, mujoco_warp 3.10.0.1, newton 1.4.0.dev0, Genesis v1.2.1). Method: seven parallel code-reading threads over fresh clones of mujoco, mujoco_warp, warp, newton, PhysX, IsaacLab, Genesis; claims verified with grep against the checkouts; web checks for governance, roadmaps, and the Genesis benchmark controversy. Charts use estimated ranges where noted; all hard numbers trace to the companion note.